CRITICAL CODE REVIEW & WEEKLY BATTLE PLAN

先让它摔不坏
再让它交得出去

Aetheris 数据平台 · 批判性代码审视与一周作战计划

26 个问题的定位、取证与修复顺序 · 2026-09-14 — 09-20 逐日排期

- 审视范围：**整个 Monorepo**
- 阻断级：**3 项**
- 高危：**7 项**
- 审视日期：**2026-09-13**

目录

- 0 · 一句话结论
- 1 · 审视方法与取证基线
- 2 · 两处必须更正的事实
- 3 · 问题总表
- 4 · 阻断级问题详解（B1–B3）
- 5 · 高危问题详解（H1–H7）
- 6 · 中危问题（M1–M11）
- 7 · 低危与仓库卫生（L1–L5）
- 8 · 下一步急需解决的东西
- 9 · 接下来一周的详细作战计划
- 10 · 一条逆耳的提醒（战略层）
- 附 · 取证命令索引

Aetheris 批判性代码审视 · 问题清单与一周作战计划

> 审视日期：2026-09-13
> 审视范围：整个 monorepo —— 10 个 Spring Boot 服务 + gateway + React 控制台 + 56 个 Flyway 迁移 + Docker Compose + CI
> 审视方法：源码逐行核查 + 命令行取证（git 跟踪状态、测试报告、依赖扫描、配置比对），**每条结论都附带文件路径与行号**
> 与既有文档的关系：`docs/roadmap.md` 记录"已完成什么"，`docs/enterprise-delivery-roadmap.md` 规划"要往哪走"，**本文件只回答一件事：现在哪里在漏、哪里会炸、下周先堵哪几个洞。**

---

0. 一句话结论

> **这套系统的工程密度已经很高，但它的"交付成熟度"严重落后于"编码密度"——442 个测试全绿、11 个容器全部健康、56 个迁移整齐排布，然而它没有一次数据库备份、没有一个指标端点、没有一条跨服务 traceId、没有一个可安装升级的发布物。**
>
> 换句话说：**它现在是一个极优秀的作品，还不是一个能交付的产品。** 下周的工作重点不该是"再加一个能力"，而是**把已经写好的能力变成"敢让别人碰"的系统**。

本次共定位 **26 个问题**：阻断级 3、高危 7、中危 11、低危 5。其中 3 个直接决定"能不能交给别人用"，7 个决定"出问题能不能查、能不能扛并发"。

---

1. 审视方法与取证基线

取证项	实测结果	命令/位置
Java 版本	21.0.7 LTS ✅	`java -version`
Maven CLI	❌ **本地已损坏**（`ClassNotFoundException: org.codehaus.plexus.classworlds.launcher.Launcher`）	`mvn -v`
后端测试存量	71 个 surefire 报告，**442 tests / 0 failures / 0 errors** ✅	`surefire-reports/TEST-*.xml`
后端测试文件	70 个	`find services apps/gateway -name "*Test.java"`
前端测试文件	19 个	`find apps/web/src -name "*.test.ts*"`
Flyway 迁移（源码）	**56 个**（metadata 18 / workflow 15 / system 5 / quality 4 / auth 4 / serving 3 / ingestion 3 / governance 2 / ai-agent 2）	`find services -name "V*__*.sql" -not -path "*/target/*"`
回滚迁移（U 前缀）	**0 个** ❌	`find . -name "U*__*.sql"`
全局异常处理器	仅 **auth、metadata 两个服务有**，其余 8 个服务 + gateway 为 0 ❌	`grep -rl ControllerAdvice`
指标采集 / 链路追踪 / 集中日志	**全部为 0** ❌	`grep micrometer|prometheus|traceId|MDC|otel`
Logback 配置	**0 个** ❌	`find -name "logback*.xml"`
备份 / 恢复脚本	**0 个** ❌	`find -iname "*backup*" -o -iname "*pg_dump*" -o -iname "*restore*"`
Compose 日志轮转	**0 处** ❌	`grep -c "max-size|logging:" docker-compose.yml`
前端 TODO/FIXME	0（Java 侧同样 0）—— 说明债务不是以标记形式存在的	`grep -rn TODO|FIXME`
前端 `any` 逃逸	14 处（无 `@ts-ignore` / `@ts-expect-error`）	`grep -rn ": any|as any"`
前端 useEffect 竞态保护	**166 处 useEffect，AbortController / cancelled 命中 0** ❌	`grep -rn "AbortController|cancelled"`
前端页面数 / 源文件数	58 个页面 / 133 个源文件	`find apps/web/src/pages -name index.tsx`

2. 两处必须更正的事实（既有报告与代码不符）

在动手之前，先把两个已经被写进文档、但与代码事实不符的结论纠正掉——否则后续排期会建立在错误的基线上。

2.1 「110 个 Flyway 迁移」→ 实际源码是 56 个

`artifacts/Aetheris-全项目审查报告.md` 第 2.3 节统计为 110 个迁移。实测源码 `services/*/src/main/resources/db/migration/` 下只有 **56 个** V 前缀迁移；差额来自 `target/classes/db/migration/` 下的构建副本被一并计入。**迁移数量被高估了近一倍**，这会影响"迁移健康度"的判断。

2.2 「前端权限是伪安全、可越权」→ 实测不构成越权

子调查给出了一条 P0 结论：前端把角色存在 localStorage（`apps/web/src/utils/access.ts:5` 的 `getCurrentRole()` 直接读 `localStorage.getItem(ROLE_KEY)`），用户改一行 DevTools 就能解锁 `/admin/*` 路由，因此是越权漏洞。

我复核后认为这个定性不准确，实测证据如下：

`apps/gateway/src/main/java/com/aetheris/gateway/config/JwtAuthFilter.java:168-181`：

```
ServerHttpRequest mutated = exchange.getRequest()
    .mutate()
    .headers(headers -> {
        headers.set("X-User-Name", username);
        headers.set("X-User-Role", authenticatedRole);    // ← 角色来自 JWT 声明，覆盖客户端传值
        headers.set(IdentitySigner.IDENTITY_HEADER, identitySignature);
    })
```

网关**用 JWT 中的角色强制覆盖** `X-User-Name / X-User-Role / X-User-Id`，客户端伪造的这些头会被重写丢弃；写操作还要经 `AuthPermissionGate` → auth 的 `/internal/permissions/may-mutate|may-manage` 判定，**auth 不可用时 fail-closed 返回 503**；用户管理接口有服务端 `AdminGuard.requireManage()`（`services/auth/ … /AuthUserAdminController.java:152`）。

所以：前端角色门禁确实只是"UI 收敛"，但不是安全漏洞——真正的边界在网关和 auth。 正确的表述是「**纵深防御不完整**」：`metadata` 有 `IdentityVerifier` 验签，而 `auth`、`system`、`ai-agent` 仍直接信任明文头（见 P2-1），一旦有内网直连路径就存在冒充空间。

保留一条真实结论：前端角色存 localStorage 且**登出时未清理 react-query 缓存**（`TopNav.tsx:74-82`），存在"切换账号后读到上一个账号缓存数据"的真实风险——这条要修。

---

3. 问题总表

编号	级别	问题	位置
B1	🔴 阻断	全平台**零备份、零恢复方案**，pgdata 单卷即全部身家	全局 / `docker-compose.yml:60,419`
B2	🔴 阻断	**可观测性三缺**：无指标、无 traceId、无集中日志；json-file 日志无上限	全部服务 pom / `application.yml`
B3	🔴 阻断	`.env.example` 的占位密钥**能通过启动校验**，照抄即上线可伪造 JWT	`.env.example:13,18,21`
H1	🟠 高	权限列表**逐行远程鉴权**，N 行 = N 次 HTTP，auth 抖动即级联挂死	`MetadataTableController.java:69,189-195`
H2	🟠 高	血缘影响分析**每跳全表加载**，最坏 500 次全表扫描	`FieldLineageService.java:37`
H3	🟠 高	语义指标列表 **N+1 查询 + 无 LIMIT**	`MetricDefinitionService.java:79-89,280-294`
H4	🟠 高	`@Transactional` 事务内发起**远程 HTTP 调用**，连接池可被拖垮	`TaskController.java:117,121,158`
H5	🟠 高	看板发布版本号**"先读后写"非原子**，并发必现 500	`AnalyticsRepository.java:116-132`
H6	🟠 高	质量规则表达式**弱校验后直接拼 SQL**，存在注入面	`QualityExecutionService.java:92-108,120-125`
H7	🟠 高	**8/10 服务无全局异常处理器**，异常直接裸奔成 500	auth、metadata 之外全部
M1	🟡 中	身份验签**不一致**：metadata 验签，auth/system/ai-agent 信任明文头	`ActivityController.java:24-36`、`ChatController.java`
M2	🟡 中	任务实例接口**缺资源级授权**，任意登录用户可枚举全量实例	`TaskController.java:254-274`
M3	🟡 中	JDBC **无连接池** + `DriverManager.setLoginTimeout` 全局静态副作用	`JdbcService.java:384-392`
M4	🟡 中	搜索/列表**在内存里 limit**，未下推数据库	`InternalAgentToolController.java:86-88`
M5	🟡 中	Flyway **零回滚能力**，迁移含 2 处不可逆 `DELETE FROM`	`V6__metadata_table_sync.sql:20`、`V8__lineage_source.sql:19,34`
M6	🟡 中	`postgres` 容器**未加固**（无 `cap_drop: ALL`），全栈无日志轮转	`docker-compose.yml:43-65`
M7	🟡 中	dev 覆盖层把前端暴露到 **0.0.0.0**	`docker-compose.dev.yml:20-21`

编号	级别	问题	位置
M8	中	CI **缺密钥扫描与源码 SBOM**；deploy 无健康校验/回滚；漏洞策略 `ignore-unfixed`	`.github/workflows/*.yml`
M9	中	前端**装而不用** react-query（全站 6 处引用），200+ 请求手写样板	`main.tsx:15`、各页面
M10	中	166 处 `useEffect` **无任何竞态/卸载保护**	前端全局
M11	中	大列表**无虚拟滚动**（75 个 Table，0 命中虚拟化）；antd chunk **1.3MB**	`vite.config.ts:38-45`
L1	低	`.git-commit-msg.tmp` **已被 git 跟踪**	仓库根
L2	低	仓库根遗留 `hs_err_pid*.log` / `replay_pid*.log` / `derby.log`，**可能含堆内存片段**	仓库根
L3	低	遗留未展开字面量目录 `${system:java.io.tmpdir}/${system:user.name}/`	仓库根
L4	低	死代码 `SectionHeader.tsx`（0 引用）+ 11 个 `comingSoon` 声明 + **无 ESLint**	`src/components/`、`navigation.tsx`
L5	低	本地 **Maven CLI 损坏**，`mvn` 完全不可用，影响开发与验收	环境

---

4. 阻断级问题详解（B1–B3）

B1. 全平台零备份 —— 这是唯一"灭顶"级风险

证据：`find . -iname "*backup*" -o -iname "*pg_dump*" -o -iname "*restore*"` 在排除 `node_modules` 后**结果为空**。数据全部落在一个命名卷上：

```
# docker-compose.yml
services:
  postgres:
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

影响：一次 `docker compose down -v`、一次卷损坏、一次勒索加密，**10 个服务的全部业务数据、权限配置、审计日志、任务历史、语义资产一次性归零**，且没有任何东西可以恢复。这不是"缺一个功能"，这是"整套系统押在一个卷上"。

还有一层叠加风险（M5）：**56 个迁移没有任何 U 前缀回滚脚本**，其中 2 个迁移带 `DELETE FROM`（`V8__lineage_source.sql:19,34` 直接删血缘数据）。升级失败时既不能回滚代码，也没有数据快照可退。

修法（详见 D1）：

1. `pg_dump -Fc` 定时全量 + WAL 归档，落到对象存储/异机；
2. 把卷换成绑定挂载或至少加独立备份卷；

3. **必须做一次"空环境恢复演练"**——恢复不是"有备份文件"，而是"从零能跑起来"；
4. 破坏性迁移前强制快照，`DELETE FROM` 改为标记失效。

B2. 可观测性三缺 —— 出故障时只能靠猜

证据（三条命令全部返回 0）：

```
grep -rn "micrometer|prometheus" services/*/pom.xml apps/gateway/pom.xml  → 0 命中
grep -rn "traceId|MDC|OpenTelemetry" services apps/gateway                → 0 命中
find . -name "logback*.xml"                                               → 0 个
grep -c "max-size|logging:" docker-compose.yml                           → 0
```

各服务 `application.yml` 的 actuator 只暴露 `health,info`，没有 `/actuator/prometheus`。日志是默认 json-file 驱动且**没有 `max-size`**，长期运行会撑爆磁盘。

影响：这套系统有 11 个容器、跨服务 HTTP 调用链（网关→workflow→metadata→数据源，最长 5 跳）、有 Spark 远端执行、有 Quartz 集群调度——**但一次慢查询无法从看板追到语义层再追到 SQL 和引擎**，一次超时无法判断卡在哪一跳，一次 P95 劣化没有任何历史数据可比。项目本身有这个自觉（`docs/enterprise-delivery-roadmap.md:90` 把 R11 可观测列为必需项），但排在 11 次迭代之后——**这个顺序是错的，可观测性应该是"现在就做"，因为后面所有修复都需要它来验证效果。**

修法（详见 D2）：加 `micrometer-registry-prometheus`，暴露 `/actuator/prometheus`；filter 里生成 `X-Request-Id` 写入 MDC 并沿调用链透传；logback 统一 JSON 格式 + traceId 字段；compose 加 `logging.options.max-size/max-file`；起一个 Prometheus + Grafana。

B3. 弱默认密钥能通过启动校验

证据：`.env.example` 给出的"示例值"，长度**恰好都越过启动校验门槛**：

变量	示例值	长度	校验门槛	结果
`JWT_SECRET`	`change-me-to-a-long-random-secret`	32 字符	`JwtAuthFilter` 要求 **≥32 字节**	✅ 通过
`INITIAL_ADMIN_PASSWORD`	`change-me-before-first-start`	28 字符	`DataInitializer` 要求 **≥12 字符**	✅ 通过
`AETHERIS_INTERNAL_TOKEN`	`change-me-internal-token`	26 字符	仅要求非空	✅ 通过

`docker-compose.yml` 用的是 `${JWT_SECRET:?}` 语法——它只校验"变量已设置"，**不校验"值不是示例值"**。

影响：按 README 的"`cp .env.example .env` → `docker compose up`"照做的用户，会得到一个 **JWT 密钥公开在 GitHub 上的、管理员口令可猜的**可运行系统。攻击者可以离线伪造任意用户的 JWT（包括 admin），而 `AETHERIS_INTERNAL_TOKEN` 泄露意味着**服务间身份签名（`X-Aetheris-Identity`）可被伪造**。

修法（详见 D3）：
1. `.env.example` 的敏感项**留空**，附带生成命令；
2. 启动时校验"值不等于示例值"，命中即拒绝启动并打印明确错误；
3. 提供一个 `make secret` / 引导脚本，一键生成全部随机密钥；
4. 顺带统一镜像标签：`.env.example:22` 是 `m1-exec-base-20260908`，`docker-compose.yml:69` 默认是 `security-20260908`——**不一致会导致拉错镜像**。

---

5. 高危问题详解（H1–H7）

H1. 权限逐行远程鉴权：一次列表请求 = N 次 HTTP

证据 `services/metadata/ ... /controller/MetadataTableController.java:69` 与 `:189-195`：

```
case OPAQUE → filteredList(request, service.list());   // ← 先把全表捞出来
...
return tables.stream()
    .filter(table → security.isAllowed(username, "table", table.getId(), "view", ...))
    .toList();                                          // ← 每张表一次远程鉴权
```

`MetadataSecurity.isAllowed` → `AuthPermissionClient` → `POST /internal/permissions/check`，**每条记录一个往返**。同样的写法在 `SemanticMetricController.java:37-43` 也存在。

影响：`OPAQUE` 模式下（用户没有任何 manifest 级授权时），一个 200 张表的列表页 = **200 次跨服务 HTTP**。而 `AuthPermissionGate` 的超时配置是 500ms 连接 + 1500ms 读（`apps/gateway/ ... /AuthPermissionGate.java:38-40`）——**auth 一旦抖动，单次请求最坏 200 × 2s = 400s，页面直接挂死，并且形成对 auth 的自我 DDoS**。这是本次审视中"改动收益最大"的一条。

修法：
1. 用 `visibility` 接口一次性取回**可见 ID 集合**，SQL 用 `IN (...)` 过滤，把 N 次网络调用压成 1 次；
2. 单请求内对 `(user, resource, id, action)` 做 request-scope 本地缓存，去重重复判定；
3. 给"逐行判定"设硬上限（超过阈值直接拒绝并要求走授权），避免无界放大。

H2. 血缘影响分析：每跳全表加载，最坏 500 次

证据 `services/metadata/ ... /service/FieldLineageService.java:37`（单行压缩写法）：

```
public List<LineageFieldRelation> impact(String table,String column){
  ...
  while(!queue.isEmpty()&&seen.size()<500){
    String key=queue.removeFirst();
    List<LineageFieldRelation> next=list(null,null).stream()      // ← list(null,null) = 全表加载
        .filter(r→r.getSourceTable().equalsIgnoreCase(parts[0])
                && r.getSourceColumn().equalsIgnoreCase(parts[1])).toList();
```

`list(null, null)` 每次迭代都把整张 `lineage_field_relation` 拉进内存再内存过滤。循环上限 500，**最坏情况 = 500 次全表扫描 + 500 个临时 List**。

影响：血缘图是"数据资产识别最后一公里"的核心能力（也正是你近期商业化的方向），但它**在真实规模下会先超时或 OOM**。表越全、血缘越密，越容易崩——**这是一个会随成功而恶化的缺陷**。

修法：按 `(source_table, source_column)` 建索引，把逐跳查询改成带 `WHERE` 的递归 CTE（PostgreSQL `WITH RECURSIVE`），一次数据库往返拿到全链路；或在应用层做逐跳 `WHERE source_table IN (...)` 的批量查询，并把 `seen` 上限做成可配置的保护阈值而非性能依赖。

H3. 语义指标列表 N+1 + 无 LIMIT

证据 `services/metadata/ ... /service/MetricDefinitionService.java:79-89`（列表路径）与 `:280-294`（`resolvePublished`）：

```
mapper.selectList(...).stream().map(this::resolvePublished)...
// resolvePublished 内部再各查一次：版本快照(jdbc.query) + 业务域(domainService.require)
```

同一个文件 `:81-88` 还显示**先全量捞出再在内存里过滤** keyword/domain，没有 `LIMIT`。

影响：列表接口的查询次数 = 指标数 × 3，且结果集随指标增长无界。语义层是这个平台对外最有价值的能力（README 花了整整一节讲它），**它的入口接口却是 N+1 + 无界**。

修法：快照表与业务域 JOIN 一次取回；keyword/domain 过滤下推 SQL；分页参数强制上限。

H4. `@Transactional` 事务内发起远程 HTTP 调用

证据 `services/workflow/ … /controller/TaskController.java:117,121` 与 `:158`：

```
@PostMapping
@Transactional                                      // ← 事务已开启
public ResponseEntity<?> create(@Valid @RequestBody Task task, …) {
    String rejection = validateDefinition(task);   // ← 内部走 POST /internal/sql/validate-write
```

`validateDefinition` → `executionClient.validateWrite(…)` 是**跨服务 HTTP**。

影响：事务横跨网络 I/O。metadata 变慢或超时时，**数据库连接被长时间占用**（持有连接的线程在等网络），连接池（`AETHERIS_DB_POOL_MAX_SIZE` 默认 8，见 `docker-compose.yml:18`）很容易被耗光，进而**整个 workflow 服务失去数据库连接**。这是典型的"小改动、大风险"。

修法：把远程校验移到事务边界**之前**（Controller 先校验，再进事务方法），事务内只做本地持久化。改动量极小。

H5. 看板发布版本号非原子 → 并发必现 500

证据 `services/serving/ … /repository/AnalyticsRepository.java:116-132`：

```
int version = dashboard.publishedVersion() + 1;                    // ← 先读
jdbc.update("INSERT INTO serving.bi_dashboard_version(…) …");    // ← 后写
```

唯一约束见 `services/serving/src/main/resources/db/migration/V1__analytics.sql:51` 的 `UNIQUE(dashboard_id, version_no)`。

影响：两个管理员（或两次重试）同时发布，都算出同一个 `version`，插入撞唯一约束抛 `DuplicateKeyException`。而 serving **恰好是 8 个没有全局异常处理器的服务之一**（H7）——**用户看到的是裸 500**，没有"版本冲突，请刷新重试"的提示。

修法：改成单语句原子递增（`version_no = (SELECT COALESCE(MAX(version_no),0)+1 FROM … WHERE dashboard_id=?)`，或 `INSERT … ON CONFLICT`），并把 `DuplicateKeyException` 映射为 **409 Conflict**。

H6. 质量规则表达式弱校验后直接拼 SQL

证据 `services/quality/ … /service/QualityExecutionService.java:92-108`：

```
return "SELECT COUNT(*) AS violation_count FROM " + target + " WHERE NOT (" + expression + ")";
```

而"安全校验"只有 `:120-125` 这一层：

```
private void rejectUnsafeExpression(String expression) {
    if (expression.contains(";") || upper.contains("--")
        || upper.contains("/*") || upper.contains("*/")) {
        throw new IllegalArgumentException("规则表达式只允许单条语句且不能包含注释");
    }
}
```

影响：只拦分号与注释，**子查询、函数调用、布尔恒等式全部放行**。例如 `NOT (1=0 OR (SELECT pg_sleep(30))::text <> '')` 之类可以构造慢查询绕过"只读"语义。`target` 有 `requireQualifiedIdentifier` 保护（`[A-Za-z_][A-Za-z0-9_$]*` 的 1–3 段），但

`expression` 没有。

缓解因素：规则需要 `quality` 的写权限才能配置，因此**不是未授权攻击面，而是"有配置权的用户可越过预期的只读约束"**。定性为高危而非阻断。

修法：复用 `SemanticMetricQueryService` 的思路——表达式走**白名单 AST 解析**（列名 + 允许的操作符 + 字面量），而不是字符黑名单；`sql` 类型分支同理，`^(SELECT|WITH)` 这个正则太弱。

H7. 8/10 服务没有全局异常处理器

证据：

```
grep -rl ControllerAdvice services/*/src  → 仅 auth、metadata
各服务统计：auth 1 / metadata 1 / system 0 / quality 0 / ingestion 0
          governance 0 / workflow 0 / ai-agent 0 / serving 0 / gateway 0
```

影响：这 8 个服务的任何未捕获异常**直接把异常消息和堆栈返回给调用方**，同时暴露内部类名/路径信息（信息泄露），并让前端只能靠 `message` 字符串猜错因。H5 的"裸 500"就是这个缺陷的直接表现。**这是一条低成本、高一致性收益的修复。**

修法：抽一个共享的 `@RestControllerAdvice` 基类（或者各服务 15 行统一实现），把异常映射成统一错误体：`{code, message, requestId}`；`DuplicateKeyException → 409`、`AccessDenied → 403`、`MethodArgumentNotValid → 400`、其余 `→ 500` 且**只回 requestId 不回堆栈**。

---

6. 中危问题（M1–M11）摘要

编号	问题	关键位置	为什么值得修
M1	身份验签不一致：`metadata` 验签，`auth` / `system` / `ai-agent` 信明文头	`ActivityController.java:24-36`（`X-User-Role` 默认 `viewer`）、`ChatController.java`（`X-User-Name` 直读）	内网直连即可冒充身份。当前端口未对外（`127.0.0.1` 绑定）所以不可利用，但**纵深防御的假设不一致**，一旦引入内网东西向流量即成缺口
M2	任务实例接口无资源级授权	`TaskController.java:254-274`（对比 `logs` / `attempts` 有 `canReadInstance`）	任意登录用户可枚举全量实例，含 `triggeredBy` 与错误信息
M3	JDBC 无连接池 + 全局静态超时	`JdbcService.java:384-392`（`DriverManager.getConnection` + `DriverManager.setLoginTimeout(8)`）	每次查询新建物理连接；`setLoginTimeout` 影响同 JVM 全部驱动，并发语义不可控
M4	内存 limit 未下推	`InternalAgentToolController.java:86-88`	表量增长后每次 agent 搜索拉全表
M5	零回滚 + 不可逆 DELETE	`V6__metadata_table_sync.sql:20`、`V8__lineage_source.sql:19,34`	与 B1 叠加放大；升级失败无退路
M6	postgres 未加固 + 无日志轮转	`docker-compose.yml:43-65`（对比 redis 有 `cap_drop: ALL`）	10 个 Java 服务与 web 都做了加固，**唯独数据库容器漏了**；全栈日志无上限
M7	dev 把前端暴露到 0.0.0.0	`docker-compose.dev.yml:20-21`	生产 compose 绑了 `127.0.0.1`（`:406`），dev 没绑
M8	CI 缺密钥扫描 / 源码 SBOM / 部署回滚	`.github/workflows/{ci,build,deploy}.yml`（trivy 均 `ignore-unfixed: true`）	误提交密钥无人拦；deploy 仅 `git pull + compose up`，无健康校验与回滚
M9	react-query 装而不用	`main.tsx:15` 配置了 QueryClient，全站仅 6 处引用	128K 的依赖换来零收益，同时 200+ 请求手写样板无缓存/去重/失效能力
M10	166 处 useEffect 无竞态保护	全前端（`AbortController` 命中 0）	快速切页/tab 切换时"旧响应覆盖新响应"必现
M11	无虚拟滚动 + antd chunk 1.3MB	`vite.config.ts:38-45`；75 个 Table	大列表全量渲染；首屏需加载约 1.5MB

7. 低危与仓库卫生（L1–L5）

```
L1  .git-commit-msg.tmp                    tracked=YES  ignored=no   ← 明确卫生缺陷
L2  hs_err_pid40904.log / hs_err_pid9678.log / replay_pid*.log / derby.log
                                           tracked=no   ignored=YES  ← 仍在磁盘，可能含堆内存片段（密钥/令牌）
L3  ${system:java.io.tmpdir}/${system:user.name}/   ← Hive 未展开变量的字面量目录
L4  SectionHeader.tsx（0 引用）、navigation.tsx 的 11 个 comingSoon、无 ESLint 配置
L5  mvn -v → ClassNotFoundException: org.codehaus.plexus.classworlds.launcher.Launcher
```

处理建议：
- `git rm --cached .git-commit-msg.tmp`，并在 `.gitignore` 补 `*.tmp`、`hs_err_pid*`、`replay_pid*`；
- 删除 4 个 JVM 转储日志（**先确认里面没有明文密钥**再删，或直接视为敏感文件彻底清除）；
- 删除字面量目录；
- 删除死代码，补 ESLint（`eslint-plugin-react-hooks` + `@typescript-eslint/no-explicit-any`）；
- **修 Maven**：这是每天都要用的工具，坏着会持续拖慢一切验证工作（`mvn test` 目前根本跑不起来，只能依赖历史 surefire 报告）。

---

8. 下一步急需解决的东西（按优先级）

第一优先：让系统"摔不坏"（本周必须完成）

1. **B1 备份 + PITR + 恢复演练** —— 唯一"灭顶"级单点，其它问题都可以慢慢来。
2. **B2 可观测性基线** —— 没有它，后面每一处修复都无法验证效果。
3. **B3 弱默认密钥闸门** —— 一行示例值的疏忽即可让生产 JWT 可伪造。

第二优先：让系统"扛得住"（本周完成，见 D4/D5）

4. **H1 权限批量判定** —— 单请求 N 次 HTTP，改动收益最大。
5. **H2 + H3 血缘与语义列表下推** —— 直接决定"数据资产识别"这条商业化主线能不能在真实规模下跑。
6. **H4 + H5 + H7** —— 事务边界外移、发布原子化、统一异常处理，三条都是低成本高确定性收益。
7. **H6 质量表达式白名单** —— 安全边界一致性。

第三优先：让系统"交得出去"（下周，见 D7）

8. **R1 实时湖仓 ADR**（不写代码，只写决策）—— 这是 `enterprise-delivery-roadmap.md` 指定的下一次迭代入口。
9. **安装/升级最小闭环设计** —— 环境检查 + 迁移 + 健康校验 + 回滚。

明确"本周不做"

不做	理由
❌ 继续扩展 M5b Git 供应链细节	`enterprise-delivery-roadmap.md:149` 自己已列为"停止加码"项，收益已递减
❌ 再做整体视觉翻新	`:147` 已列为停止项；当前设计足够进入试点
❌ 为 11 个 `comingSoon` 补占位页面	`:148` 已列为停止项；没有真实执行/权限/审计的页面是负资产
❌ 引入 Flink / Iceberg 代码	R1 只产出 ADR + BOM + allowlist，不写业务代码
❌ 每次小改都全量重打 11 个镜像	`:153` 已列为停止项；日常只跑受影响模块测试

---

9. 接下来一周的详细作战计划

> 周期：**2026-09-14（周一）– 2026-09-20（周日）**，7 天
> 单人 + AI 辅助，每天按"上午做实现、下午做验证、收尾写记录"的节奏
> **每天的完成定义（DoD）统一为**：代码改动 + 一条可复制粘贴的验收命令（能贴出真实输出）+ 一条追加到 `.workbuddy/memory/YYYY-MM-DD.md` 的记录

总览

日	主题	交付物	验收命令
D1 9/14 一	🔴 数据保命：备份与恢复	`scripts/pg-backup.sh`、`scripts/pg-restore.sh`、恢复演练记录	空环境恢复后 `docker compose ps` 全 healthy
D2 9/15 二	🔴 可观测基线	metrics 端点 + traceId 透传 + logback JSON + 日志轮转	`curl /actuator/prometheus` 有指标；日志含同一 requestId
D3 9/16 三	🔴 上线安全闸门	密钥强制校验 + 一键生成脚本 + postgres 加固 + 端口收敛	用示例值启动 → **拒绝启动**；用随机值 → 正常
D4 9/17 四	🟠 性能止血 A（权限与语义层）	批量权限判定 + 语义列表 JOIN/分页	200 表列表页鉴权调用次数从 N → 1
D5 9/18 五	🟠 性能止血 B（血缘、事务、异常）	递归 CTE + 事务外移 + 原子版本 + 统一异常体	血缘影响分析 1 次查询；并发发布返回 409 非 500
D6 9/19 六	🟠 质量与前端护栏	表达式白名单 + ESLint + 竞态修复 + 死代码清理	`tsc --noEmit` + `vitest run` + `eslint` 全绿
D7 9/20 日	🎯 方向决策与交付基线	R1 ADR 初稿 + 安装/升级最小闭环设计 + 周复盘	文档评审通过；下一迭代范围冻结

D1 · 9/14（周一）· 数据保命：备份与恢复 🔴

为什么排第一：这是唯一"灭顶"级风险。所有其它工作都建立在"数据还在"这个前提上。

任务清单

1. **写备份脚本** `scripts/pg-backup.sh`
   - `pg_dump -Fc -Z6` 全量自定义格式，落到独立目录/卷，文件名带时间戳；
   - 保留策略：最近 7 份日备 + 4 份周备（先手工清理，不要一上来就写复杂的保留逻辑）；
   - 备份完成后**校验可读性**（`pg_restore --list` 能列出内容才算成功），失败时非零退出。
2. **写恢复脚本** `scripts/pg-restore.sh`
   - `dropdb` + `createdb` + `pg_restore`；
   - **明确警告这是破坏性操作**，要求显式传 `--yes` 才执行。
3. **加一条 Makefile 目标**：`make backup` / `make restore FILE=...`。
4. **做一次真实恢复演练（关键）**
   - 先备份 → 起一个全新的临时 compose 项目名（`-p aetheris-drill`）→ 从空库恢复 → 验证业务可用；
   - 验证点：能登录、`metadata.metadata_table` 行数一致、`workflow.task_instance` 行数一致、前端能打开工作台。
   - **把演练结果（含行数比对）写进 `docs/` 作为第一份恢复报告。**
5. **破坏性迁移前置快照**：在 `V8` 这类带 `DELETE FROM` 的迁移说明中补一句"执行前必须备份"，并在 README 的升级章节写入。
6. （可选，若时间允许）**WAL 归档**：`archive_mode=on` + `archive_command` 到独立目录，把 RPO 从"最后一次 dump"压到分钟级。

DoD / 验收

```
make backup                                  # 退出码 0，产出 .dump 且 pg_restore --list 可读
make restore FILE=<最新备份> --yes            # 空环境恢复成功
docker compose -p aetheris-drill ps          # 全部 healthy
# 行数比对
psql -c "select count(*) from metadata.metadata_table"   # 与备份前一致
```

风险提示：不要再犯"备份了但从没恢复过"的经典错误——D1 的产出**不是脚本，是"从零恢复成功"这个事实**。

D2 · 9/15（周二）· 可观测性基线 🔴

为什么排第二：D4–D6 的每一处性能与稳定性修复，都需要指标来证明"修好了"，否则只能靠感觉。

任务清单

1. **指标**：各服务 `pom.xml` 加 `micrometer-registry-prometheus`；`application.yml` 把 actuator 暴露从 `health,info` 扩到 `health,info,prometheus`；确认 `/actuator/prometheus` 返回文本格式指标。**注意**：该端点不能经网关对外暴露（保持在容器网络内），Prometheus 在 compose 内网抓取。
2. **traceId 透传**：
   - 网关 filter 生成 `X-Request-Id`（无则新建，有则沿用），写入响应头；
   - 各服务加一个 `OncePerRequestFilter`，把 `X-Request-Id` 放进 `MDC`，并在响应头回写；
   - 内部 HTTP 客户端（`executionClient`、`AuthPermissionClient`、`MetadataQueryClient`）统一带上该头 → **一条链路上 5 跳共享同一个 requestId**。
3. **结构化日志**：新增统一的 `logback-spring.xml`（放 `services/*/src/main/resources/` 或抽公共模块），JSON 单行输出，字段含 `timestamp / level / service / requestId / user / logger / message`；**去掉所有** `printStackTrace`（当前为 0，保持住）。
4. **日志轮转**：`docker-compose.yml` 给全部服务加：
   `yaml logging: driver: json-file options: max-size: "20m" max-file: "5"`
5. **起观测栈**：compose 加 `prometheus` + `grafana`（放独立 profile `obs`，**不污染默认环境**）；建一张最小看板：各服务 QPS / P95 / 5xx 比例 / 连接池占用 / JVM 堆。

DoD / 验收

```
curl -s localhost:8081/actuator/prometheus | head        # 有指标输出
# 触发一次跨服务调用（如新建任务），在后端日志中查同一 requestId 是否贯穿网关→workflow→metadata
docker compose logs workflow --tail 50 | grep requestId
docker inspect <container> --format '{{.HostConfig.LogConfig}}'   # max-size 已生效
```

提醒：这一步的价值不在"有 Grafana 看板"，而在**从此每次修复都有前后对比的 P95 数据**。

D3 · 9/16（周三）· 上线安全闸门 🔴

为什么：让"照抄 .env.example"不再等于"上一个 JWT 可伪造的系统"。

任务清单

1. `.env.example` **改造**：敏感项全部**留空**并附生成命令注释：
   `dotenv # 生成：openssl rand -base64 48 JWT_SECRET= # 生成：openssl rand -base64 32 AETHERIS_DATASOURCE_MASTER_KEY= # 生成：openssl rand -hex 32 AETHERIS_INTERNAL_TOKEN= # 生成：openssl rand -base64 24 INITIAL_ADMIN_PASSWORD=`

2. **启动校验**：写一个 `SecretsValidator`（各服务共享或放 gateway/auth）：
   - 拒绝 `change-me*` / `replace-with*` / `generate-a-*` / 空值；
   - 拒绝 JWT_SECRET 长度 < 32 字节；
   - 命中时**打印明确的中文错误并拒绝启动**（不要静默降级）。
3. **一键生成**：`scripts/gen-secrets.sh`（或 `make secrets`），把结果写到 `.env`（不覆盖已有值），并在 stdout 打印每个变量的用途。
4. **镜像标签统一**：`.env.example:22` 与 `docker-compose.yml:69` 对齐为同一个 tag（建议改为语义化版本如 `0.11.0`，避免日期 tag 漂移）。
5. **postgres 容器加固**：补 `cap_drop: [ALL]`（按需 `cap_add`），与 redis 及其它服务一致（M6）。
6. **端口收敛**：`docker-compose.dev.yml:20-21` 加 `127.0.0.1:` 前缀（M7）。
7. **修 Maven**（L5）：这是每天要用的工具，坏着会持续拖慢验证。

DoD / 验收

```
cp .env.example .env && docker compose up -d api-gateway
# 期望：启动失败，日志明确提示 "JWT_SECRET 不能使用示例值，请运行 make secrets"
bash scripts/gen-secrets.sh && docker compose up -d   # 期望：全部 healthy
git grep -n "change-me" -- .env.example               # 期望：仅出现在注释中
```

D4 · 9/17（周四）· 性能止血 A：权限与语义层 🟠

任务清单

1. **H1 批量权限判定**（本日主攻，收益最大）
   - 在 auth 侧补一个批量端点：`POST /internal/permissions/visibility-batch`，入参 `(username, resourceType, action, candidateIds[])`，出参**允许的 ID 集合**；
   - `MetadataTableController.java:69` 的 `OPAQUE` 分支改为：先取可见 ID 集合 → `LambdaQueryWrapper.in(id, allowedIds)` 直接 SQL 过滤，**彻底删掉** `filteredList` **逐行循环**；
   - `SemanticMetricController.java:37-43` 同样改造；
   - 加请求级去重缓存（同一请求内相同 `(resource,id,action)` 只问一次）。
2. **给逐行判定设熔断**：若某分支仍必须逐行判定，`isAllowed` 调用次数超过阈值（如 200）直接抛异常，**防止无界放大**。
3. **H3 语义指标列表**
   - `MetricDefinitionService.java:79-89`：版本快照与业务域改 JOIN 一次取回；
   - keyword/domain 过滤**下推 SQL**，加 `LIMIT` 与分页上限（建议默认 20、硬上限 200）。
4. **顺带 M4**：`InternalAgentToolController.java:86-88` 的 `.stream().limit()` 改为数据库 `LIMIT`。

DoD / 验收

- 造一张 200 张表的测试数据集，在 `OPAQUE` 模式下打一次列表接口；
- 用 D2 的指标或日志验证：**auth 的 `/internal/permissions/check` 调用次数从约 200 次降到 1 次**；
- 语义指标列表：开启 MyBatis SQL 日志，确认**指标数 N 对应固定次数查询（与 N 无关）**；
- 加回归测试：一条断言"列表接口在 OPAQUE 模式下不产生逐行远程调用"。

D5 · 9/18（周五）· 性能止血 B：血缘、事务、异常 🟠

任务清单

1. **H2 血缘影响分析**
   - 新增索引：`lineage_field_relation(source_table, source_column)`；

- `FieldLineageService.impact()` 改为 **PostgreSQL** `WITH RECURSIVE`，一次往返取全链路；
- `seen` 的 500 上限保留，但降级为"保护阈值"并在日志中告警（而不是性能依赖）。

2. **H4 事务边界外移**
   - `TaskController.java:117/121/158`：`validateDefinition(task)`（含远程 `validateWrite`）移到 `@Transactional` 方法**之前**；
   - 同类问题顺手扫一遍其它 Controller（搜 `@Transactional` + 远程客户端调用的组合）。
3. **H5 看板发布原子化**
   - `AnalyticsRepository.java:116-132` 改为单语句原子递增版本号；
   - 捕获 `DuplicateKeyException` → 返回 **409 + 明确中文提示**。
4. **H7 统一异常处理（8 个服务）**
   - 抽公共 `ApiExceptionHandler`（或各服务独立实现，保持 15 行内），统一错误体 `{code, message, requestId}`；
   - 映射规则：`MethodArgumentNotValid → 400`、`AccessDenied → 403`、`DuplicateKey → 409`、其余 `→ 500` 且只回 `requestId`；
   - **严禁把异常堆栈或内部类名返回给调用方**。
5. **M2 任务实例授权**：`TaskController.java:254-274` 的两个实例列表接口补 `canReadInstance` / owner 过滤。

DoD / 验收

```
# 血缘：对一个有多级下游的字段跑影响分析，日志中 SQL 执行次数应为 1（CTE）
# 发布并发：两个并发请求发布同一看板 → 一个 200，另一个 409（而不是 500）
# 异常：故意触发一个未预期异常 → 响应体为 {code,message,requestId}，无堆栈
mvn -pl services/workflow,services/serving,services/metadata test   # 全绿（需先修复 mvn）
```

D6 · 9/19（周六）· 质量规则与前端护栏 🟠

任务清单

1. **H6 质量规则表达式白名单化**
   - 用 AST/词法解析替代 `rejectUnsafeExpression` 的字符黑名单：只允许 `列名 / 字面量 / 比较操作符 / AND OR NOT / 括号 / IN / IS NULL / BETWEEN`；
   - 拒绝任何 `SELECT`、函数调用与子查询（`sql` 类型分支单独走"仅允许单条 SELECT 且复用 metadata 的只读守卫"）；
   - 加负向测试用例：`pg_sleep`、子查询、恒等式、注释绕过各一条。
2. **前端 M9 决策（不做全套改造，只做取舍）**
   - 二选一：**(a)** 把 3–5 个最重的列表页迁到 `useQuery`（拿缓存/失效/去重）；**(b)** 卸载 react-query，统一到手写 `api/` 层并补齐取消能力。**推荐 (b)** —— 当前收益/成本比更好，且能减 128K。
   - 无论选哪个，**登出必须清空缓存**（`TopNav.tsx:74-82` 的 `handleLogout` 加 `queryClient.clear()`，并给 `catch {}` 补用户提示）。
3. **M10 竞态修复（只修高频路径）**
   - 给 5–8 个最常切换的数据列表页加 `AbortController` 或 `cancelled` 标志（不动 166 处全部，聚焦高频）；
   - 封装一个 `useAsyncEffect` hook 供后续复用。
4. **L4 卫生**
   - 删 `SectionHeader.tsx`（0 引用）；
   - 加 ESLint：`eslint-plugin-react-hooks`（`exhaustive-deps`）+ `@typescript-eslint/no-explicit-any`（先设为 `warn`，避免一次爆红）+ `no-unused-vars`；
   - `package.json` 补 `"lint": "eslint src --ext .ts,.tsx"`；
   - `src/utils/format.ts` 统一时间/金额格式，替换页面里混用的 `toLocaleString()` / `dayjs().format()` 三种写法（先修 5 个主要页面）。

5. **L1/L2/L3 仓库卫生**
   - `git rm --cached .git-commit-msg.tmp`；
   - `.gitignore` 补 `*.tmp`、`hs_err_pid*`、`replay_pid*`；
   - 删除根目录 4 个 JVM 转储与 `${system:java.io.tmpdir}` 字面量目录（**删前确认无明文密钥**）。

DoD / 验收

```
cd apps/web && npm run type-check && npm test && npm run build && npx eslint src
# 期望：tsc 0 错误；vitest 全绿；build 成功；eslint 无 error（warn 可接受）
git status --short          # 期望：无 .git-commit-msg.tmp；无 hs_err/replay 残留
```

D7 • 9/20（周日）• 方向决策与交付基线 🎯

为什么留一天给"想"：这一周前 6 天都在"堵洞"。如果不留时间回答"接下来做什么"，下周会本能地继续加功能——而 `enterprise-delivery-roadmap.md:143-153` 已经明确列出了 7 项"停止继续加码"的工作。

任务清单

1. **R1 的 ADR 初稿**（只写决策，不写代码）
   - 写清 Flink / Flink CDC / Iceberg / REST Catalog / 对象存储 / 候选 Trino 各自的**职责边界与否决理由**；
   - 产出**版本 BOM 表**：JDK / Flink / Flink CDC / Iceberg runtime / Spark / Trino / 对象存储 SDK / 各数据库驱动，逐项标注"候选版本 + 兼容性验证状态"；
   - 产出 **connector allowlist**：来源 URL + SHA-256 + 许可证 + CVE 状态（哪怕先只填 1–2 行，把结构立起来）；
   - **硬门槛写进文档**：R1 的验收标准是"同一张 Iceberg v2 表可被 Flink 写、Spark 与 Trino 读"，**不是"Flink 容器能启动"**。
2. **安装/升级最小闭环设计**（一页纸）
   - 环境检查器（版本、端口、磁盘、内存）→ 配置向导（密钥生成）→ 迁移执行 → **健康校验** → 失败回滚；
   - 明确当前最大的交付阻碍：**没有"全新安装/N-1 升级/失败回滚"三件事的可重复验证**。
3. **周复盘**
   - 用 D2 的指标给 D4/D5 的修复出**前后对比数据**（P95、远程调用次数、查询次数）；
   - 更新 `docs/roadmap.md` 与 `CHANGELOG.md`（本轮的修复按 Keep a Changelog 记入 Unreleased）；
   - 冻结下一迭代范围（R1 的 6 个可提交增量，见 `enterprise-delivery-roadmap.md:189-197`）。
4. **修正两处既有文档事实错误**（见本报告第 2 节）
   - `artifacts/Aetheris-全项目审查报告.md:94` 的"110 个迁移"改为 56；
   - 第 5.5 节关于前端权限的表述改为"UI 收敛 + 纵深防御不一致（非越权）"。

DoD / 验收

- `docs/adr-0001-realtime-lakehouse.md` 存在且包含职责边界 + BOM + allowlist 结构 + 硬门槛；
- `docs/install-upgrade-design.md` 存在（一页纸）；
- `CHANGELOG.md` 的 Unreleased 段落记录本周 7 天改动；
- 下一迭代的 6 个增量写进 roadmap 并有明确退出门槛。

一周节奏与纪律

纪律	说明
上午实现 / 下午验证	每个改动**当天**跑出验收输出，不留"待验证"
每天收尾写日志	追加到 `.workbuddy/memory/YYYY-MM-DD.md`，含验收命令与真实输出
单主题提交	每天一个可回滚的提交，提交信息用 `fix:` / `feat:` / `docs:` 前缀（项目现有风格）
不碰无关文件	遵守 `enterprise-delivery-roadmap.md:183` 的第 10 条：不覆盖用户未纳入本次范围的文件
只跑受影响模块测试	日常 `mvn -pl <module> test` + 前端 `npm test`；**只有 D7 才做全量构建与 Compose 升级验证**
卡住就降级	某个修复超时（如 WAL 归档），当天降级为"记录方案 + 下一迭代再做"，不整周偏移

本周产出清单

```
scripts/pg-backup.sh                备份
scripts/pg-restore.sh               恢复
scripts/gen-secrets.sh              密钥一键生成
scripts/（Makefile 目标）            make backup / restore / secrets
services/*/src/ ... /SecretsValidator 启动密钥闸门
services/*/logback-spring.xml       结构化日志
docker-compose.yml                  observability profile + 日志轮转 + postgres 加固
docs/backup-restore-drill-*.md      恢复演练报告
docs/adr-0001-realtime-lakehouse.md R1 架构决策
docs/install-upgrade-design.md      安装/升级最小闭环设计
CHANGELOG.md / docs/roadmap.md      本轮记录与范围冻结
```

---

10. 一条逆耳的提醒（战略层）

代码层面我挑出了 26 个问题，但**最大的风险不在代码里**。

这个项目现在的状态是：**5.4 万行手写代码、442 个全绿测试、11 个健康容器、56 个整齐的迁移、10 个微服务的完整闭环**——工程密度在国内个人项目里属于极罕见的一档。但同时：

- 没有一次备份；
- 没有一个指标端点；
- 没有一条跨服务 traceId；
- 没有一个能被别人按文档安装起来的发布物；
- 语义层最强的能力（跨表 JOIN、派生指标）**仍在"已知边界"里没做**；
- 而 `docs/roadmap.md` 里 M5b 这样的供应链细节**已经做到了"过度精密"的程度**。

这说明过去一段时间的投入分配，偏向"证明我能做难的事"，而不是"让别人能用起来"。 对"1–2 个月内产生现金流"这个目标来说，前者的边际收益正在快速下降。

结合你正在考虑的"数据资产识别最后一公里"定位（记为 H2），我的判断是：

1. **能力上你已经够了**——元数据、血缘（表级 + 字段级）、业务域、语义指标、质量、权限、审计，**"数据资产识别"需要的零件基本齐全**。缺的不是功能，是**规模化下的性能（H2/H3 就是直接拦路的两个）和可交付性（B1/B2/B3）**。
2. **所以本周这 7 天不是"技术债清偿"，而是"把资产变成可变现形态"的必要前置**——一个连备份都没有、慢查询查不到原因的系统，无论定位多好，都过不了任何一个客户的采购技术评审。
3. **下周之后的方向取舍**，建议在 D7 明确写成文档：**是继续推进 R1 实时湖仓（12–20 个月的 GA 长路），还是先做"数据资产识别"的最小可交付版本（安装包 + 资产盘点报告 + 血缘影响分析），用 1–2 个月验证能否换回现金流。** 这两条路的排期不冲突，但**资源的先后**

顺序必须现在定，否则会继续"哪儿难做哪儿"。

附录：本次审视的取证命令索引

```
# 仓库与历史
git log --oneline -40 ; git status --short ; git branch -a
git ls-files --error-unmatch .git-commit-msg.tmp        # 确认被跟踪
git check-ignore -v .env hs_err_pid40904.log            # 确认被忽略

# 测试与构建
find . -name "TEST-*.xml" -path "*surefire*" | wc -l    # 71
grep -h -o 'tests="[0-9]*"' ... | awk '{s+=$1} END{print s}'   # 442 / 0 / 0
mvn -v                                                   # 复现 Maven 损坏

# 代码异味
grep -rn "TODO\|FIXME\|HACK" --include=*.java services apps   # 0
grep -rn ": any\|as any\|@ts-ignore" apps/web/src             # 14
grep -rn "catch (Exception" --include=*.java services apps    # 25
grep -rn "printStackTrace" --include=*.java services apps     # 0（良好）

# 可观测性与运维
grep -rn "micrometer\|prometheus" services/*/pom.xml apps/gateway/pom.xml   # 0
grep -rn "traceId\|MDC\|OpenTelemetry" services apps/gateway                # 0
find . -name "logback*.xml"                                                 # 0
find . -iname "*backup*" -o -iname "*pg_dump*" -o -iname "*restore*"        # 0
grep -c "max-size\|logging:" docker-compose.yml                             # 0
grep -rl "@RestControllerAdvice\|@ControllerAdvice" services apps           # 2（auth, metadata）

# 迁移
find services -name "V*__*.sql" -not -path "*/target/*" | wc -l             # 56
find . -name "U*__*.sql" -not -path "*/target/*" | wc -l                    # 0
grep -rl "DELETE FROM\|DROP TABLE\|DROP COLUMN" --include="V*.sql" services # 2

# 权限链（用于推翻"越权"结论）
sed -n '168,181p' apps/gateway/src/main/java/com/aetheris/gateway/config/JwtAuthFilter.java
sed -n '60,75p;185,196p'
services/metadata/src/main/java/com/aetheris/metadata/controller/MetadataTableController.java
grep -n "requireManage"
services/auth/src/main/java/com/aetheris/auth/controller/AuthUserAdminController.java
```

Aetheris · 让数据有序流动，让价值清晰发生。